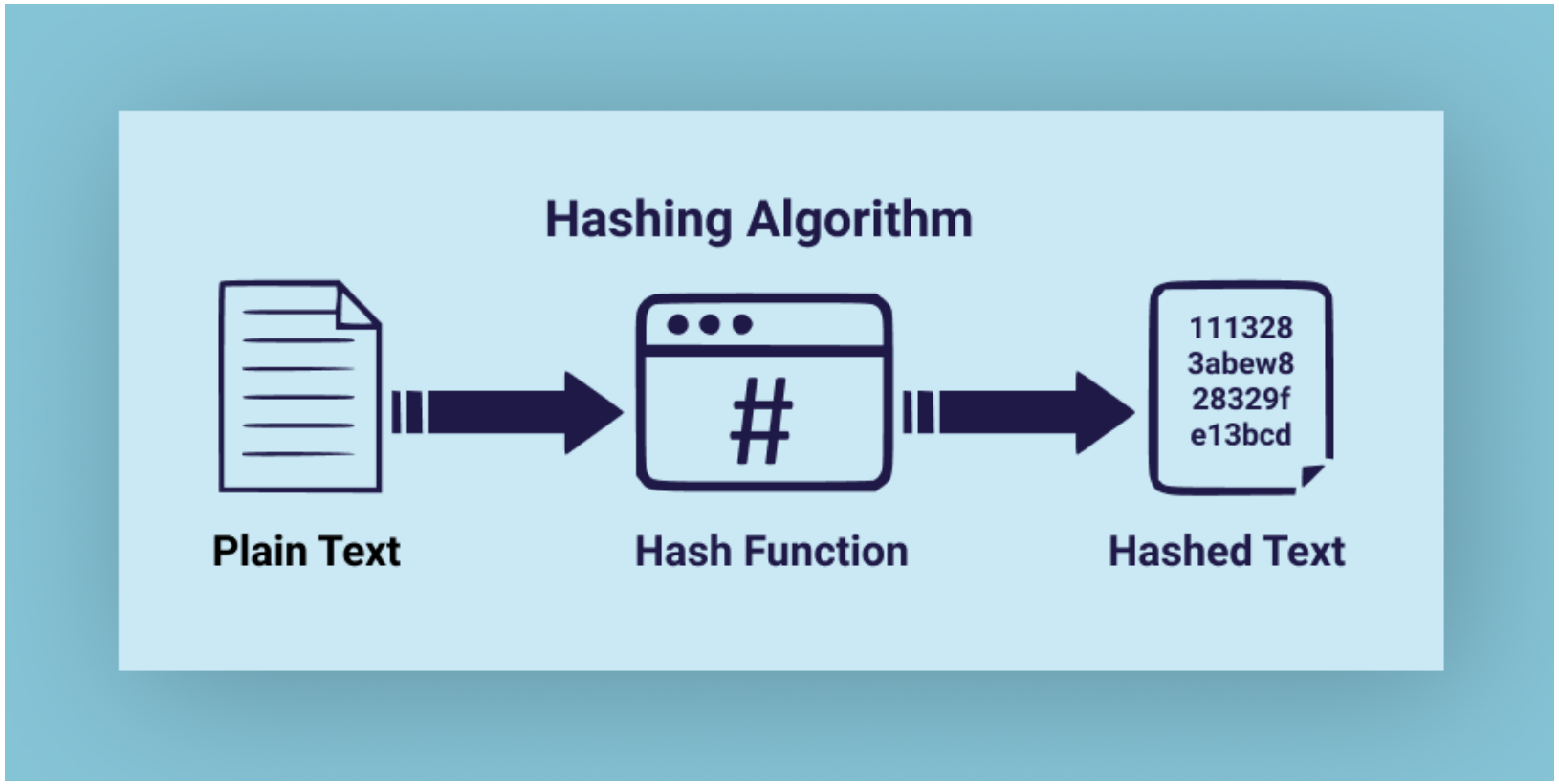
Data Structures, In Detail

CS 374

Hashing/Dictionaries

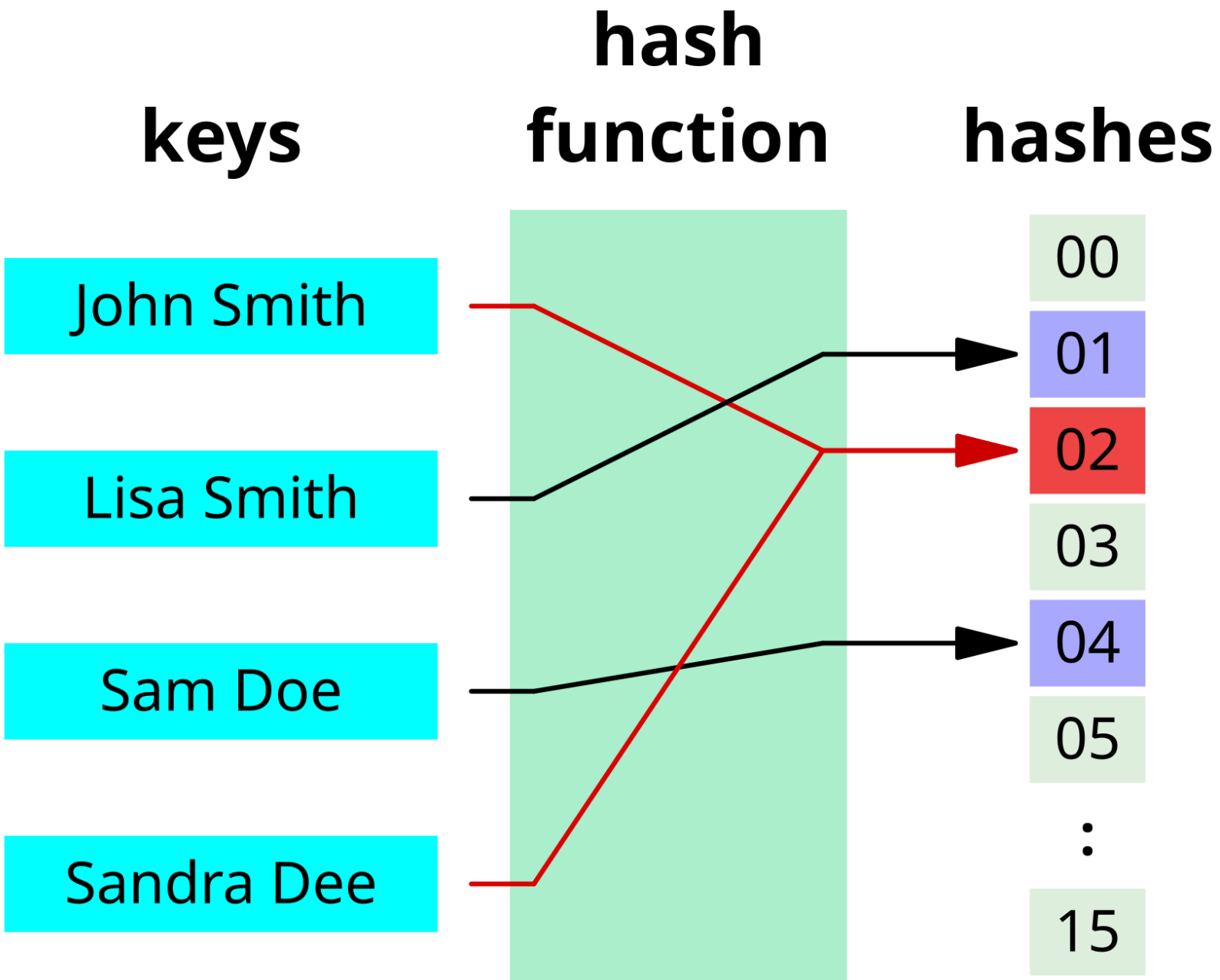

Hashing Algorithm
#
111328
3abew8
28329f
e13bcd
Plain Text
Hash Function
Hashed Text

Hashing

- A *hash* is a function that maps keys to an integer.
- Used in dictionaries, etc. to help you find it
- Also used for cryptography

- Have a function, then mod by m to stick it into spot m in the structure

- Collisions are an issue – what if two things have same hash?


keys
hash function
hashes
John Smith
Lisa Smith
Sam Doe
Sandra Dee
00
01
02
03
04
05
:
15

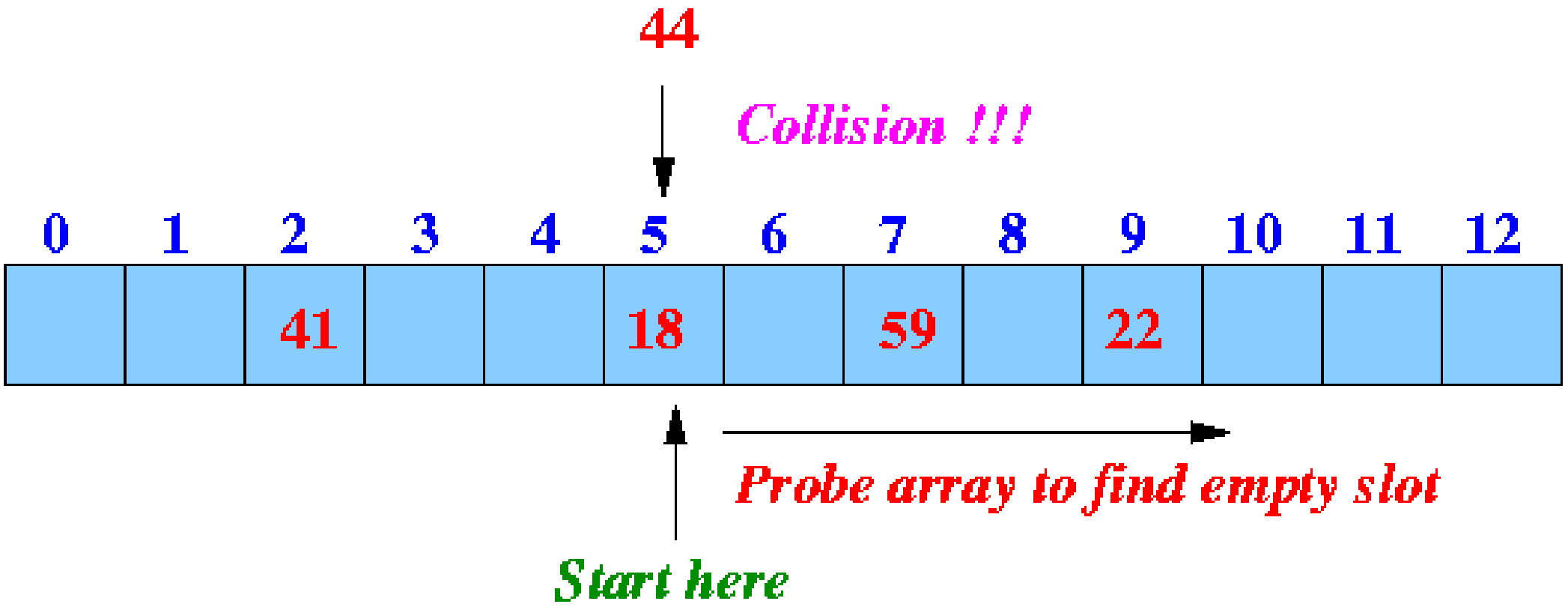
Example

- 3. Given a hash function of $f(x) = x + 31 \bmod 11$, determine *if* there is a collision for items inserted with the keys 60, 9, 45, 3, 70, 5, 90, 93, 39, 77, 70 in that order.
- At which item does the first collision occur?

0	1	2	3	4	5	6	7	8	9
34	5	55	21		2		3	8	13

Figure 3.11: Collision resolution by open addressing and sequential probing, after inserting the first eight Fibonacci numbers in increasing order with $H(x) = (2x + 1) \mod 10$. *The red elements have been bumped to the first open slot after the desired location.*

Linear Probing



Example

- 3. Given a hash function of $f(x) = x + 31 \bmod 11$, determine *if* there is a collision for items inserted with the keys 60, 9, 45, 3, 70, 5, 90, 93, 39, 77, 70 in that order.
- What is the result of the array when resolving collisions using linear/sequential probing? Use arrows/different colors to demonstrate when probing occurs.

Chaining

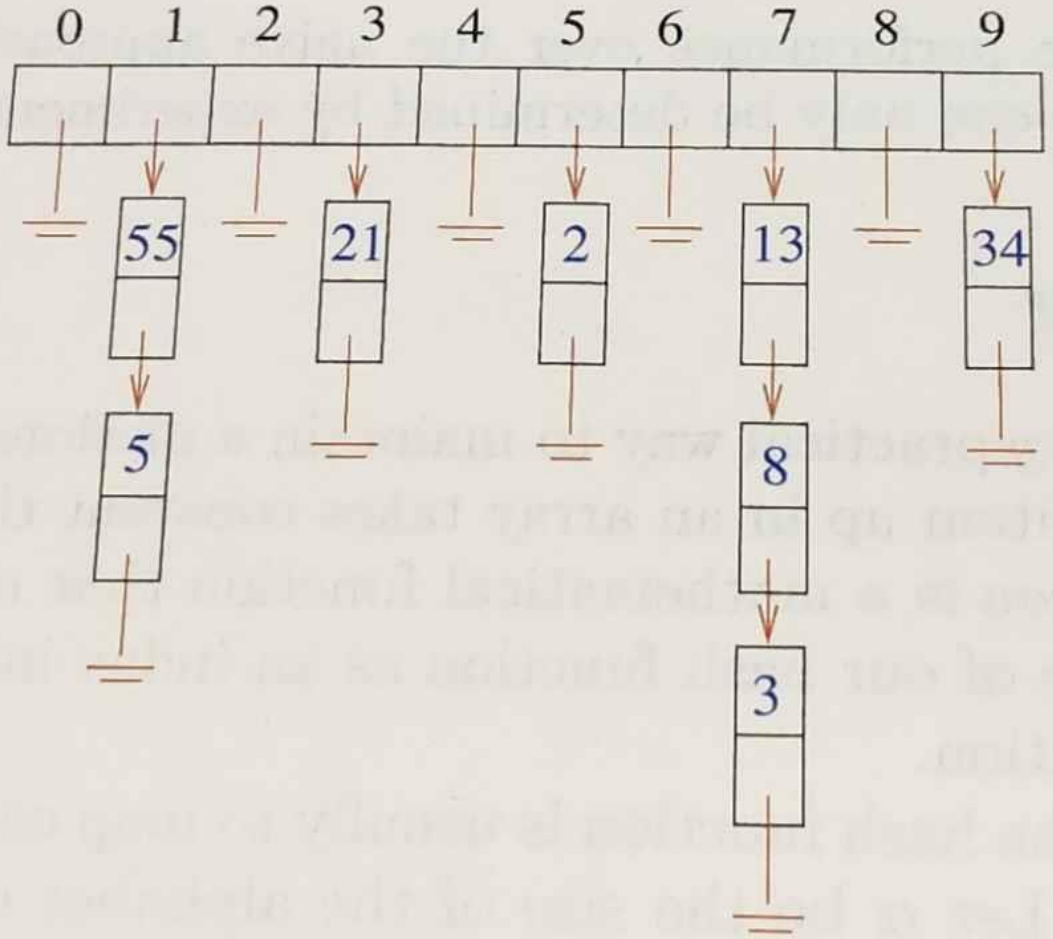
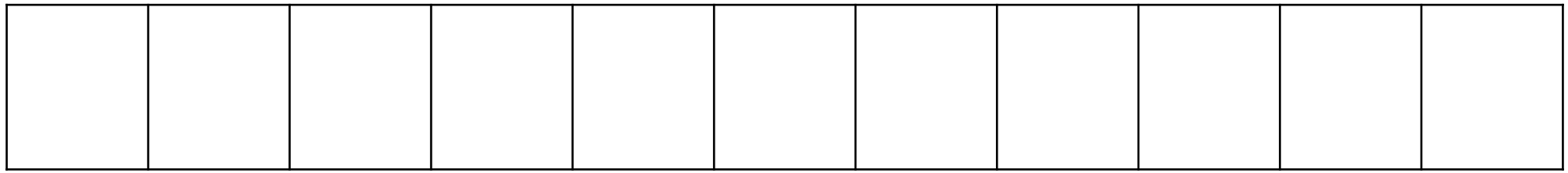


Figure 3.10: Collision resolution by chaining, after hashing the first eight Fibonacci numbers in increasing order, with hash function $H(x) = (2x + 1) \bmod 10$. *Insertions occur at the head of each list in this figure.*

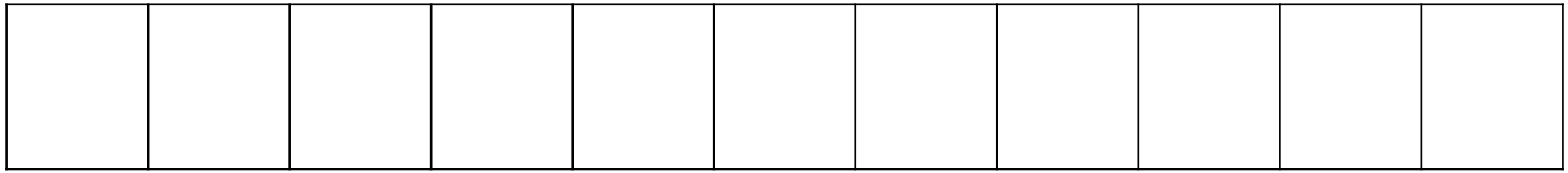
Example

- 3. Given a hash function of $f(x) = x + 31 \bmod 11$, determine *if* there is a collision for items inserted with the keys 60, 9, 45, 3, 70, 5, 90, 93, 39, 77, 70 in that order.
- What is the result of the array when resolving collisions using chaining? Draw a picture.

$f(x) = x + 31 \bmod 11$: 60, 9, 45, 3, 70, 5, 90, 93, 39, 77, 70



$f(x) = x + 31 \bmod 11$: 60, 9, 45, 3, 70, 5, 90, 93, 39, 77, 70



More Collision Resolution

- Quadratic hashing
  - If value x matches to y,
  - Instead of moving y+1, move $y+1^{2}$.
  - Then, if $y+1^{2}$ is full, try $y+2^{2}$
  - Chances are, you'll find an empty spot faster.

- Double hashing
  - Have a separate hash function that you run when there are collisions

- And more!

How many things are in there?

- Hash table has fixed size
  - Like if implemented with arrays
- So, need to keep track of "how full" it is
- Load factor: $\alpha = \frac{n}{m}$
  - n = size of data (num. entries)
  - m = number of buckets (capacity)

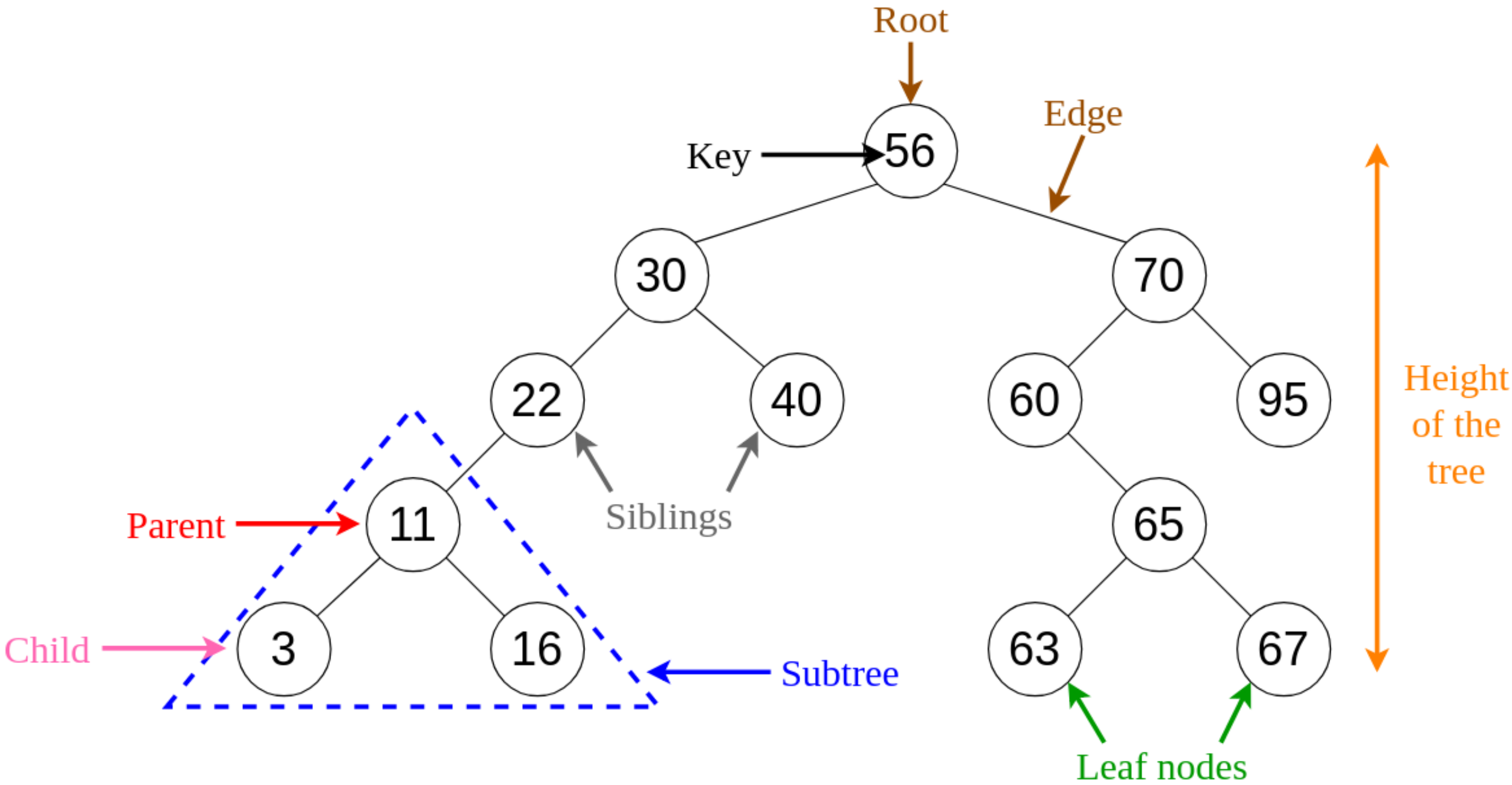
Load factor

- You decide how full you want it to get
  - Set some α_{max} threshold
- Resize, *then rehash* when it gets too big (α_{max}), or when gets too small (α_{max}/4 ??)
- Also may depend on method of collision resolution

Hash Table Operations

	Hash table (expected)	Hash table (worst case)
Search(L, k)	$O(n/m)$	$O(n)$
Insert(L, x)	$O(1)$	$O(1)$
Delete(L, x)	$O(1)$	$O(1)$
Successor(L, x)	$O(n+m)$	$O(n+m)$
Predecessor(L, x)	$O(n+m)$	$O(n+m)$
Minimum(L)	$O(n+m)$	$O(n+m)$
Maximum(L)	$O(n+m)$	$O(n+m)$

Binary Search Trees


Root
Edge
Key
56
30
70
22
40
60
95
65
11
3
16
63
67
Parent
Siblings
Child
Subtree
Height of the tree
Leaf nodes

Insert the following items into a BST:
60, 9, 45, 3, 70, 5, 90, 93, 39, 77, 70

Tree A

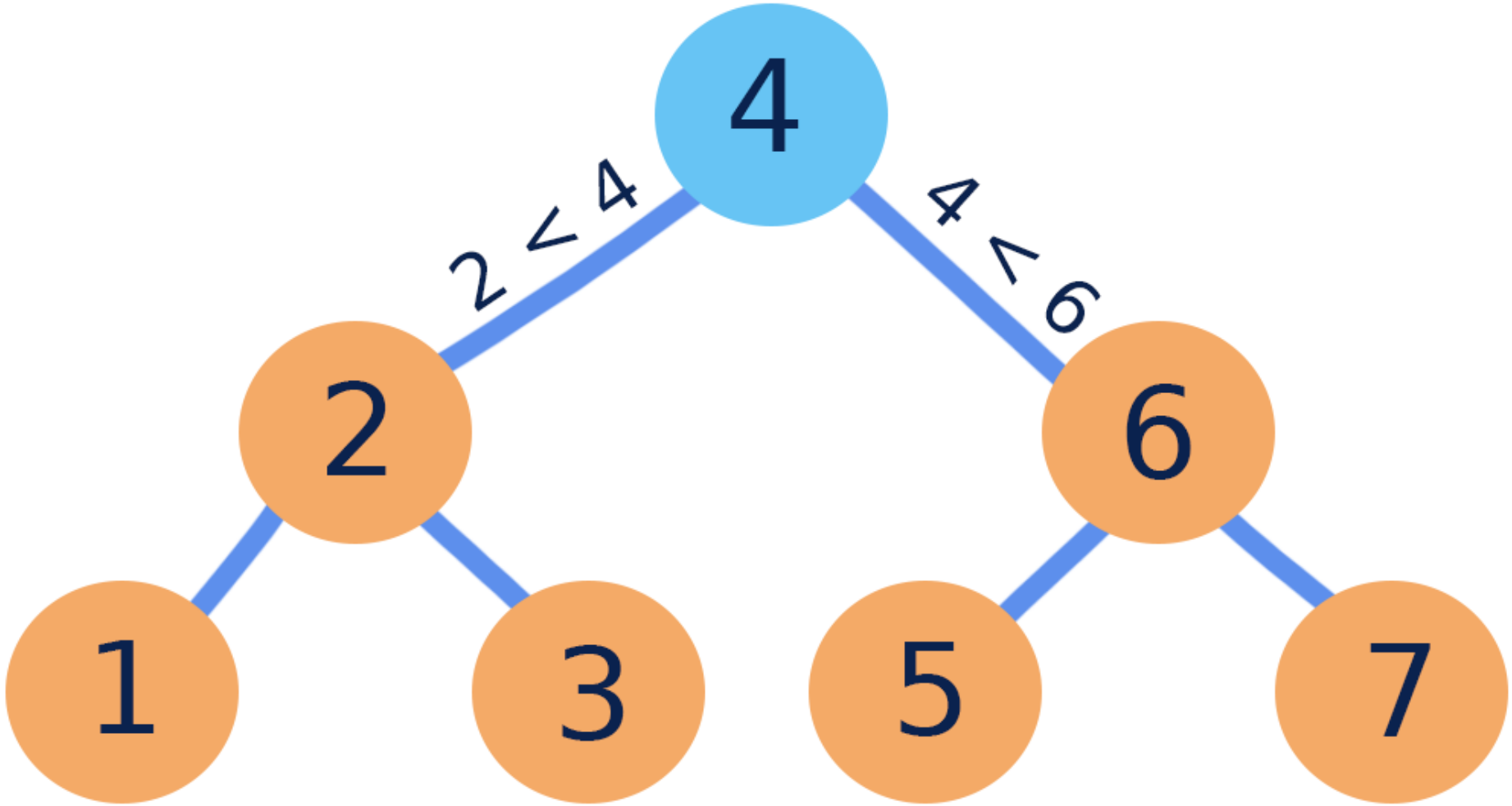
Insert the following items into a BST:

58 47 84 39 55 14 98 53 79 42 87 3 90 1

Tree B

BST Terms

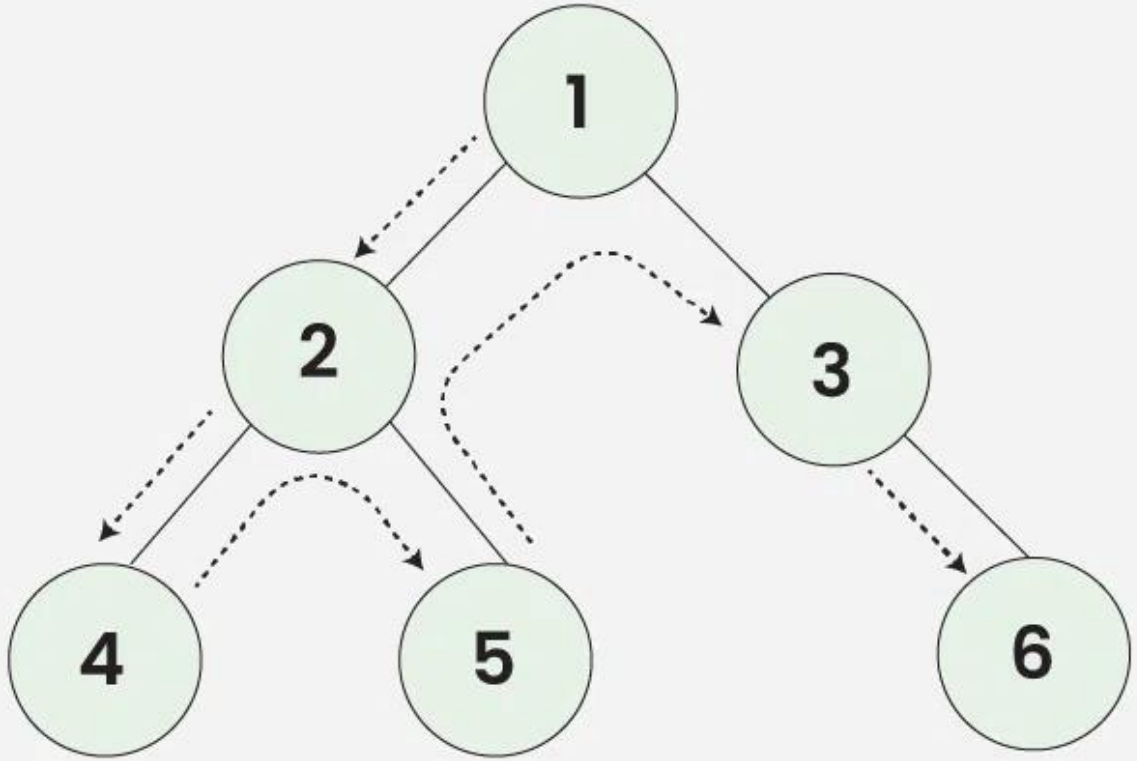
- Complete
  - All levels full
- Balanced
  - One side not more than other



In Order Traversal: 1 2 3 4 5 6 7

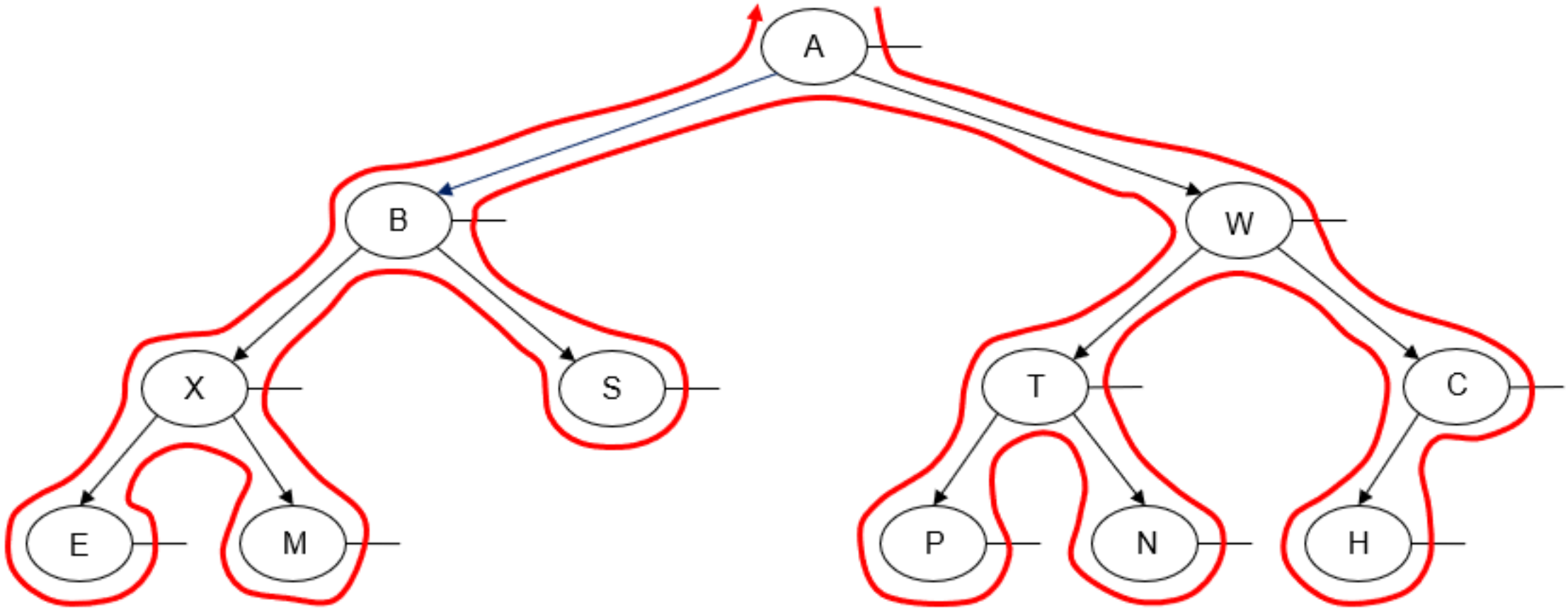
InOrder Traversal of tree A

Preorder Traversal of Binary Tree



Preorder Traversal: 1 → 2 → 4 → 5 → 3 → 6

PreOrder Traversal of tree A



- Post-order traversal: E M X S B P N T H C W A
- (this isn't a BST, just a BT)

PostOrder Traversal of tree A

BST Operations

- Search – O(h), where h= height of tree
- Find min/max – O(h)
- Traversal – O(n)
- Insertion – O(h)
- Deletion – O(h)

Warm-up
Meet up with your teams. On the whiteboard:

Problem A.

- Using *mod* 6,
- Create a hash table for 10, 20, 7, 11, 17, 14, 2, 4, 6, 15, 3
- Equation:
  - $f(x) = 18x^6 - x^4 + 4x^3 + 21x + 2$
- Rehash when $\alpha > 0.6$. What do you choose for new mod? Why?
- Use your choice of collision resolution

Problem B.

- Create a binary search tree for 20, 10, 7, 11, 17, 14, 2, 4, 6, 15, 3
- Create a binary search tree for 17, 10, 7, 11, 20, 14, 2, 4, 6, 15, 3
- These are the same numbers: What's the difference?

Heaps

Heaps

- Min-heap: smaller = at the top
- Max-heap: larger = at the top

- Complete binary tree
  - All levels are filled…
  - …except maybe the last one
  - Last level gets filled Left to Right.

Heap – Insertion

- Insert latest item in bottommost, leftmost position.
- "Bubble up" if necessary.
  - Swap the item with its parent until heap property is not violated
  - Also called "heapify", sometimes "heapify up"

Example: Insert the following numbers into a min-heap.
18, 12, 3, 7, 15, 4, 11, 16, 5, 14.
Then draw the array.

Heap – Deletion

- Remove the item at the root
- Move the "last" thing in the heap to the root
- "Bubble down" if necessary.
  - Check the children.
  - Swap the item with the smaller of the children
  - Continue swapping item down, so the smaller thing gets swapped up.
  - Also called "heapify", sometimes "heapify down"

Remove 2 things from the preceding heap.

Heaps – that's great, but how does it code???

- Heaps use binary trees
- Stored in an array with level order
  - Index 0=minimum
  - For a given index i,
    - Parent node is located at $\frac{\mathrm{i}-2}{2}$
    - Left child is located at $2i + 1$
    - Right child is located at $2i + 2$

Heap – Search

- Ehhh not really
- Used to find the *min* element
- So don't worry about it!

What do we use heaps for?

- Priority queues!
- Min-heap removal always gives the minimum element.

Heap Complexity

Draw the max-heap for the following numbers, then remove 3 elements.
3, 14, 4, 8, 15, 16, 9, 12, 20

Cool-Down: Solve my to-do list

Task	Deadline	Dr. Roscoe's Priority
Grade Algorithms HW 1	Tuesday	1
Knit hat	March 31	5
Read a research paper	Friday	2
Post the Stats homework	5pm today	2
Submit Jan Term proposal	Sunday 11:59pm	1
Post Algorithms video	5pm today	1
Write a section of research paper	5pm today	1

- Discuss on what variable or combination of variables you will optimize.
- Describe on the board how you will construct a priority queue, and use a heap to return the order of tasks I should complete, until the queue is empty.

Warm-up: Does heap insert order matter?

- Insert the following lists of numbers into a heap. Do they produce the same heap?

A. 19, 27, 13, 26, 11, 24, 18, 10, 1, 30, 28, 14, 5, 15, 4, 8, 12, 23

B. 11, 1, 8, 5, 10, 24, 14, 18, 26, 4, 19, 13, 15, 30, 27, 23, 28, 12

C. 24, 27, 11, 4, 8, 5, 1, 23, 12, 30, 18, 28, 19, 15, 14, 10, 13, 26

Union-Find / Disjoint Sets

Outline

- Analogy
- Trees
- In arrays
- Code
- Optimizations

Let's start a company

- You're wondering why I've brought you all here today...
  - Everyone is working on something different.
  - You get a number 1-26 to represent your various projects/expertises

Let's start a company

- Hang on, some of you are working on similar things.
- I will put you in different "departments"
  - pairs/groups of 3

Someone has to be in charge

- Between you, designate someone to be your "manager".
- Even if you have a direct supervisor, you still have a manager.
- When I ask you which group you're part of, respond with the name of your ultimate manager.
  - This operation is called **find()**

Merging Departments

- Merging 2 departments:
  - One of the two existing managers gets to be ultimate manager
  - Can't have 2 managers
  - The result is 1 department with 1 manager.

- This operation is called **union().**
- Specifically,

Union-Find / Disjoint Sets

- A data structure that performs 2 main operations:
  - Union(x,y)
    - Combine two departments based on any two people in those departments.
  - Find(x)
    - Return who x's ultimate manager is.

Union-Find uses a Forest

- Forest = group of un-connected trees
- Each tree in the forest is a connected group
  - A department, identified by its ultimate manager
- Merging departments means combining trees under a common root

Union Find: that's great, but how does it code???

- Remind me: What's an element of an array?

- Remind me: What's an index of an array?

Union Find: that's great, but how does it code???

- Each *element* corresponds to an array *index*.
  - Space: O(n)
- Elements not nice integers?
  - Use a Hashmap/dictionary. Map elements to integers.
- The *item* at each element is the name of the "manager"
  - If 5's root is 2, the element at index 5 is 2

Code

V 1.0 Initial Version

```
class UnionFind:
    def __init__(self, n):
        self.parent = [i for i in range(n)]

    def find(self, x):
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, x, y):
        root_x = self.find(x)
        root_y = self.find(y)
        if root_x != root_y:
            self.parent[root_y] = root_x
```

From https://www.youtube.com/watch?v=92UpvDXc8fs

Example: with 10-element array

- Break ties with a preference for smaller numbers.
- Union(5, 9)
- Union(3, 1)
- Union(8, 6)
- Union(9, 2)
- Union(4, 7)
- Union(8, 0)
- Union(7, 1)
- Union(9, 4)
- Union(2, 3)
- Union(6, 5)
- What is find(5)?

It's not perfect

- Haphazard union-ing could result in a linear structure

Making it better – By Rank / By Size

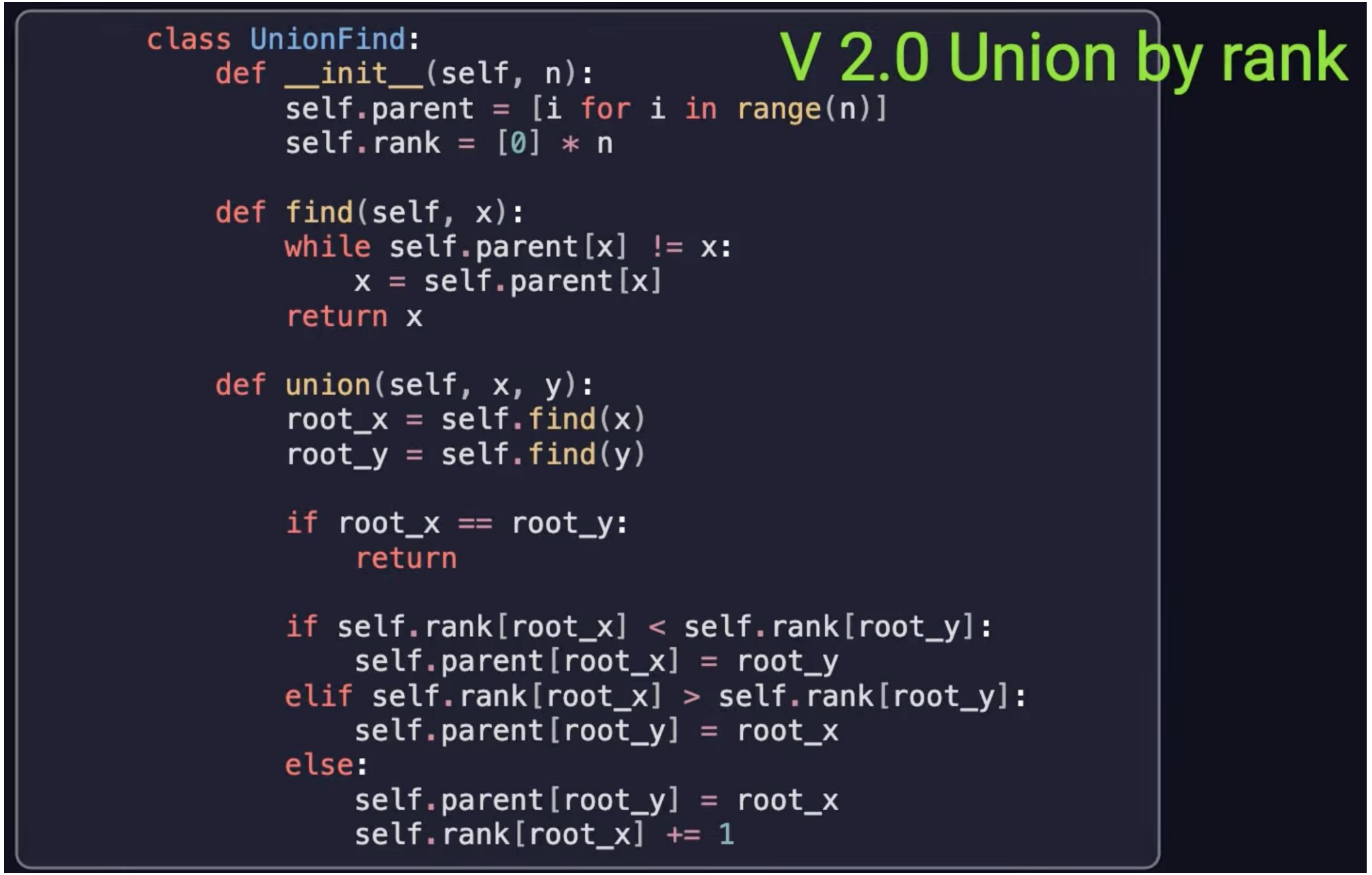
- Union by …
  - Rank = height of the tree
  - Size = number of elements in the tree
- Keep track of this in a separate array
  - Only need to keep track of the root's rank or size

Making it better – Path Compression

- When finding, if we aren't directly reporting to the manager, make it so we do directly report to the manager.
  - Redraw edges of the tree

- Makes the tree wide, but fast lookup.

Code version 2



From https://www.youtube.com/watch?v=92UpvDXc8fs

Complexity of Union-Find

- Just path compression: $\Theta(n + f \cdot (1 + \log_{2+f/n} n))$
- Just union-by-rank, no path compression: $\Theta(m \log n)$
- Union-by-rank WITH path compression: $\Theta(\alpha(n))$
  - α here is the Inverse Ackermann function
    - Ackermann function grows *really really fast*
    - Inverse means it grows slowly
    - Essentially <=4 for any quantifiable n.

- So, not actually constant time, but essentially free.

Applications

- Finding a minimum spanning tree
  - Kruskal's MST algorithm
- How many connected components of a graph are there?

- Easy-to-implement and efficient graph algorithms have wide-reaching effects.

Cool-Down

- In your teams, visualize a sequence of Union-Find operations as an array and as a forest. Use by-rank and path optimization. Break ties with a preference for smaller numbers.
    - Union(7,1)
    - Union(4,2)
    - Union(8,5)
    - Union(0,3)
    - Union(6,9)
    - Union(2,7)
    - Union(5,9)
    - Union(8,1)
    - Union(4,6)
- What is Find(4)?

Cool-Down

- Complete "Number of Provinces" LeetCode problem using UnionFind
  - https://leetcode.com/problems/number-of-provinces

Further Exploration

- https://en.wikipedia.org/wiki/Disjoint-set_data_structure#
- https://en.wikipedia.org/wiki/Ackermann_function#Inverse
- https://www.youtube.com/watch?v=92UpvDXc8fs
- https://www.youtube.com/watch?v=8f1XPm4WOUc

These slides are work-in-progress; more will be added